Multi-Vendor E-Commerce Platform API

Project Report: Spring Boot REST API Capstone (architecture, database schema, Git conventions and full ticket backlog)

1. Project Overview

The Multi-Vendor E-Commerce Platform is a backend REST API for a marketplace in the style of Amazon or Etsy. Independent merchants manage their own catalog and stock, while customers fill one cart with items from many merchants and pay in a single checkout. The system splits that checkout into one fulfillment order per merchant, tracks each order through a strict state machine, reserves stock to prevent overselling, and cleans up abandoned orders automatically. Administrators moderate the platform and see platform-wide metrics.

This project is the capstone of a Spring Boot REST API course. Every topic of the course (foundations, web layer, persistence, architecture, cross-cutting concerns and production concerns) is applied in one realistic system.

1.1 Goals

- Deliver a production-style API with clean layered architecture, not a toy CRUD app.
- Show correct handling of hard problems: concurrent stock reservation, multi-merchant order splitting, idempotent payments, and scheduled recovery.
- Be portfolio-ready: documented schema, consistent Git history, automated tests, Docker packaging and live API docs.

1.2 Scope

In scope: authentication and role-based access, merchant onboarding, hierarchical catalog, search with caching, inventory reservation, cart, multi-vendor checkout, simulated payment, fulfillment lifecycle, async notifications, scheduled cleanup, audit logging, admin moderation and metrics.

Out of scope: real payment providers, real email delivery, file or image upload, a frontend, refunds accounting, shipping-rate calculation.

1.3 User Roles

Role	Capabilities	Restrictions
CUSTOMER	Browse and search, manage cart, checkout, pay, track and cancel own pending orders	Cannot see other customers or any merchant back-office data
MERCHANT	Create and update own products and stock, view and process orders containing own items	Cannot read or modify another merchant data; needs admin approval before listing products
ADMIN	Global read/write: moderate products, manage users and merchants, cancel orders, resolve disputes, view metrics and audit logs	Every admin action is audit-logged

2. Course Coverage Map

This table shows where each course topic is exercised, so a reviewer can see the whole course is covered. Ticket IDs refer to the backlog in section 9.

Course topic	How the project uses it	Tickets
1.1 Anatomy and structure	Package-by-feature layout, main class, starters	ECOM-1
1.2 DI and bean lifecycle	Constructor injection everywhere, stateless singleton services, custom beans (clock, payment gateway, executor)	ECOM-26, ECOM-34, ECOM-40
1.3 Configuration	Profiles (dev, test, prod), typed properties for JWT, reservation timeout, scheduler cron	ECOM-2
2.1 REST controllers	Resource-oriented endpoints, path variables, query params, pagination	ECOM-8, ECOM-18, ECOM-20, ECOM-21

Course topic	How the project uses it	Tickets
2.2 Request/response, DTOs, Jackson	Request/response records, JSON formatting of money and timestamps	ECOM-8, ECOM-18, ECOM-32
2.3 Validation	Bean Validation on all inputs, custom validators (price, tracking number, status transition)	ECOM-8, ECOM-18, ECOM-28, ECOM-38
2.4 Exception handling	One @ControllerAdvice, uniform error body, domain exceptions	ECOM-4
3.1 JPA and entities	14 entities, one-to-many, many-to-one, self-reference (categories)	ECOM-7, ECOM-16, ECOM-18, ECOM-30
3.2 Spring Data repositories	Derived queries, @Query for dashboards and the scheduler scan, Specifications or filters for search	ECOM-21, ECOM-36, ECOM-42
3.3 Transactions	Atomic checkout, payment commit, order cancellation with stock release	ECOM-26, ECOM-31, ECOM-35
4.1 Layered architecture	Controller to Service to Repository with strict rules (section 4)	all
4.2 DTOs vs entities	Entities never leave the service layer; mappers convert	ECOM-8, ECOM-32
5.1 Logging	SLF4J, correlation id in MDC, structured key=value logs for business events	ECOM-5
5.2 Testing	Unit, @WebMvcTest, @DataJpaTest, @SpringBootTest end-to-end	ECOM-50 to ECOM-53
5.3 Security basics	Filter chain, BCrypt, method security, ownership checks	ECOM-9, ECOM-11
5.4 JWT auth	Stateless login, token filter, role claims	ECOM-10, ECOM-11

Course topic	How the project uses it	Tickets
6.1 Actuator	Health, metrics, info, custom health indicator	ECOM-54
6.2 AOP	@Audited aspect for price changes and cancellations, timing aspect	ECOM-44, ECOM-45
6.3 Caching	@Cacheable popular searches, eviction on product change	ECOM-22
6.4 Async and scheduling	@Async notifications, @Scheduled cleanup every 15 minutes	ECOM-40 to ECOM-43
7.1 Packaging and Docker	Multi-stage Dockerfile, docker-compose with PostgreSQL	ECOM-3, ECOM-56
7.2 End-to-end walkthrough	Documented checkout flow from HTTP request to DB rows and back	ECOM-53, ECOM-57

3. Technology Stack

Area	Choice	Note
Language / runtime	Java 21	LTS
Framework	Spring Boot 3.x (Web, Data JPA, Validation, Security, Actuator, Cache, AOP)	Course core
Database	PostgreSQL 16	H2 only for quick local experiments
Migrations	Flyway	Bonus: schema versioned in Git, never auto-generated in prod
Auth	Spring Security + JWT (JJWT library)	Stateless
Cache	Caffeine via Spring Cache	In-memory, simple to explain
Mapping	Manual mapper classes or MapStruct	Pick one and stay consistent

Area	Choice	Note
Testing	JUnit 5, Mockito, Testcontainers (PostgreSQL)	Testcontainers is a bonus
API docs	springdoc-openapi (Swagger UI)	Bonus
Packaging	Maven, Docker, docker-compose	
CI	GitHub Actions	Build and test on every push and PR

4. Architecture

4.1 Layered Architecture

Requests flow Controller to Service to Repository to Database. Each layer has one responsibility:

- **Controller:** HTTP only. Parses input, triggers validation, calls exactly one service method, returns a DTO with the correct status code. No business rules, no repositories.
- **Service:** All business rules and transaction boundaries (@Transactional lives here). Works with entities internally and returns DTOs or domain results. Services of other features are called through their service, never through their repository.
- **Repository:** Data access only. No business decisions.
- **Entity:** Persistence model, never serialized to JSON.
- **DTO:** Request and response records with validation annotations; the only types visible to the outside world.

4.2 Package Structure (package by feature)

```
com.example.marketplace
├── MarketplaceApplication.java
├── common/            # error handling, base entity, audit annotation,
config, utils
├── security/          # JWT service, filter, SecurityConfig, CurrentUser
helper
├── user/              # User, controller, service, repository, dto
├── merchant/          # MerchantProfile, approval flow
├── catalog/           # Category, Product, search, storefront
├── inventory/         # Inventory, InventoryReservation, InventoryService
├── cart/              # Cart, CartItem
├── order/             # Order, FulfillmentOrder, OrderItem, state machine
├── payment/           # Payment, PaymentGateway interface + simulated impl
├── notification/      # Notification, async sender
├── audit/             # AuditLog, @Audited, AuditAspect
├── admin/             # moderation, disputes, metrics
└── scheduler/         # abandoned-order cleanup job
```

Inside each feature package: `XController`, `XService`, `XRepository`, `X` (entity), `dto/`, `XMapper`.

4.3 Cross-Cutting Design Decisions

- **Money:** BigDecimal with scale 2, stored as NUMERIC(12,2). Never double.
- **Time:** Instant in code and TIMESTAMPTZ in the database; an injectable Clock bean makes the 15-minute rule testable.
- **Identifiers:** BIGINT identity keys internally; human-friendly order_number exposed to customers.
- **Snapshots:** Order items copy product title and unit price at purchase time so later price edits never change history.
- **Idempotency:** Payment requests carry an idempotency key; repeating a request never charges or commits stock twice.
- **Pagination:** All list endpoints accept page, size and sort and return a consistent page envelope.
- **Error format:** Every error returns timestamp, status, code, message, path and optional field errors.

5. Domain Rules

5.1 Order State Machine

The customer-level Order has the states PENDING, PAID and CANCELLED. Each merchant-specific FulfillmentOrder follows the full lifecycle below. Illegal transitions throw a domain exception mapped to HTTP 409.

From	To	Triggered by	Side effects
(none)	PENDING	Customer checkout	Stock reserved, expires_at set to now + 15 minutes
PENDING	PAID	Successful payment	Reservations committed, stock permanently deducted, confirmation notification queued
PENDING	CANCELLED	Customer, admin or scheduler	Reservations released, stock restored
PAID	PROCESSING	Owning merchant	Audit entry
PROCESSING	SHIPPED	Owning merchant	Tracking number required, customer notified
SHIPPED	DELIVERED	Merchant or admin	Delivered timestamp set
PAID or PROCESSING	CANCELLED	Admin only	Audit entry, customer notified, stock policy documented in ECOM-47

A fulfillment order cannot move once it is DELIVERED or CANCELLED.

5.2 Inventory Rules

- Each product has available_quantity (can be sold) and reserved_quantity (held by pending orders).
- **Reserve:** available decreases and reserved increases, in one atomic statement that fails if available is too low.
- **Commit (on payment):** reserved decreases; the units are gone for good.
- **Release (on cancel or timeout):** reserved decreases and available increases.

- Invariant: available_quantity and reserved_quantity never go below zero (enforced by a CHECK constraint and by the conditional update).
- Preferred implementation: UPDATE inventory SET available_quantity = available_quantity - :q, reserved_quantity = reserved_quantity + :q WHERE product_id = :id AND available_quantity >= :q, and treat zero updated rows as out of stock. When reserving several products, process them sorted by product id to avoid deadlocks. Alternative: pessimistic write lock via @Lock.

5.3 Checkout Algorithm

1. Load the customer cart; reject if empty.
2. In a single transaction, reserve each line (section 5.2). Any failure rolls everything back and returns a 409 listing the unavailable products.
3. Group lines by merchant and create one Order plus one FulfillmentOrder per merchant, with OrderItem snapshots.
4. Create inventory_reservations rows (ACTIVE) linked to the order.
5. Clear the cart, set expires_at, return the order.
6. Payment (ticket ECOM-34/35): on success mark order and fulfillment orders PAID and reservations COMMITTED; on failure leave PENDING.

5.4 Cart Recovery Job

Every 15 minutes the scheduler selects orders with status PENDING and expires_at in the past, then cancels each one in its own transaction (so one failure does not block the rest), releases its ACTIVE reservations, writes an audit entry with actor SYSTEM and queues a notification.

6. Database Schema

Database: PostgreSQL. Enumerations are stored as VARCHAR with CHECK constraints (JPA @Enumerated(STRING)). All tables have created_at; mutable tables also have updated_at. TIMESTAMPTZ is used for all timestamps.

6.1 Tables

users

Column	Type	Constraints / notes
id	BIGINT	PK, identity
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(100)	NOT NULL, BCrypt
full_name	VARCHAR(120)	NOT NULL
role	VARCHAR(20)	CUSTOMER, MERCHANT, ADMIN
status	VARCHAR(20)	ACTIVE, SUSPENDED
created_at, updated_at	TIMESTAMPTZ	NOT NULL

merchant_profiles

Column	Type	Constraints / notes
id	BIGINT	PK
user_id	BIGINT	FK users, UNIQUE (one profile per user)
store_name	VARCHAR(120)	UNIQUE, NOT NULL
description	TEXT	
status	VARCHAR(20)	PENDING, APPROVED, REJECTED
reviewed_by	BIGINT	FK users, nullable
created_at	TIMESTAMPTZ	

categories

Column	Type	Constraints / notes
id	BIGINT	PK
name	VARCHAR(100)	NOT NULL
slug	VARCHAR(120)	UNIQUE
parent_id	BIGINT	FK categories, NULL for root categories (self-reference)

products

Column	Type	Constraints / notes
id	BIGINT	PK
merchant_id	BIGINT	FK merchant_profiles, NOT NULL
category_id	BIGINT	FK categories, NOT NULL
title	VARCHAR(200)	NOT NULL
description	TEXT	
price	NUMERIC(12,2)	CHECK price > 0
status	VARCHAR(20)	ACTIVE, HIDDEN, REMOVED (REMOVED = taken down by admin)
created_at, updated_at	TIMESTAMPTZ	

inventory (1:1 with products)

Column	Type	Constraints / notes
product_id	BIGINT	PK and FK products
available_quantity	INT	CHECK >= 0
reserved_quantity	INT	CHECK >= 0
version	BIGINT	optimistic lock (@Version)

Column	Type	Constraints / notes
updated_at	TIMESTAMPTZ	

carts and **cart_items**

Table	Column	Type	Constraints / notes
carts	id	BIGINT	PK
carts	customer_id	BIGINT	FK users, UNIQUE
carts	updated_at	TIMESTAMPTZ	
cart_items	id	BIGINT	PK
cart_items	cart_id	BIGINT	FK carts, ON DELETE CASCADE
cart_items	product_id	BIGINT	FK products
cart_items	quantity	INT	CHECK > 0; UNIQUE (cart_id, product_id)

orders (customer-level checkout)

Column	Type	Constraints / notes
id	BIGINT	PK
order_number	VARCHAR(30)	UNIQUE, public identifier
customer_id	BIGINT	FK users
status	VARCHAR(20)	PENDING, PAID, CANCELLED
total_amount	NUMERIC(12,2)	
expires_at	TIMESTAMPTZ	payment deadline (created_at + 15 min)
cancel_reason	VARCHAR(30)	nullable: CUSTOMER, ADMIN, PAYMENT_TIMEOUT
created_at, updated_at	TIMESTAMPTZ	

fulfillment_orders (one per merchant per order)

Column	Type	Constraints / notes
id	BIGINT	PK
order_id	BIGINT	FK orders
merchant_id	BIGINT	FK merchant_profiles
status	VARCHAR(20)	PENDING, PAID, PROCESSING, SHIPPED, DELIVERED, CANCELLED
subtotal	NUMERIC(12,2)	
tracking_number	VARCHAR(100)	required when SHIPPED
carrier	VARCHAR(60)	nullable
shipped_at, delivered_at	TIMESTAMPTZ	nullable
created_at, updated_at	TIMESTAMPTZ	

order_items

Column	Type	Constraints / notes
id	BIGINT	PK
fulfillment_order_id	BIGINT	FK fulfillment_orders
product_id	BIGINT	FK products
product_title	VARCHAR(200)	snapshot
unit_price	NUMERIC(12,2)	snapshot
quantity	INT	CHECK > 0

inventory_reservations

Column	Type	Constraints / notes
id	BIGINT	PK
order_id	BIGINT	FK orders

Column	Type	Constraints / notes
product_id	BIGINT	FK products
quantity	INT	CHECK > 0
status	VARCHAR(20)	ACTIVE, COMMITTED, RELEASED
created_at, resolved_at	TIMESTAMPTZ	resolved_at nullable

payments

Column	Type	Constraints / notes
id	BIGINT	PK
order_id	BIGINT	FK orders
amount	NUMERIC(12,2)	
status	VARCHAR(20)	INITIATED, SUCCEEDED, FAILED
provider_reference	VARCHAR(100)	simulated gateway reference
idempotency_key	VARCHAR(100)	UNIQUE
failure_reason	VARCHAR(200)	nullable
created_at	TIMESTAMPTZ	

audit_logs (append-only)

Column	Type	Constraints / notes
id	BIGINT	PK
actor_id	BIGINT	FK users, NULL when actor is SYSTEM
actor_role	VARCHAR(20)	CUSTOMER, MERCHANT, ADMIN, SYSTEM
action	VARCHAR(50)	e.g. PRODUCT_PRICE_CHANGED, ORDER_CANCELLED
entity_type	VARCHAR(50)	
entity_id	BIGINT	

Column	Type	Constraints / notes
old_value, new_value	JSONB	
created_at	TIMESTAMPTZ	no UPDATE or DELETE ever issued

notifications

Column	Type	Constraints / notes
id	BIGINT	PK
user_id	BIGINT	FK users
type	VARCHAR(40)	ORDER_CONFIRMATION, ORDER_SHIPPED, ORDER_CANCELLED, ...
subject	VARCHAR(200)	
body	TEXT	
status	VARCHAR(20)	QUEUED, SENT, FAILED
created_at, sent_at	TIMESTAMPTZ	sent_at nullable

disputes

Column	Type	Constraints / notes
id	BIGINT	PK
fulfillment_order_id	BIGINT	FK fulfillment_orders
raised_by	BIGINT	FK users
reason	TEXT	NOT NULL
status	VARCHAR(20)	OPEN, RESOLVED
resolution_note	TEXT	nullable
resolved_by	BIGINT	FK users, nullable
created_at, resolved_at	TIMESTAMPTZ	

6.2 Relationships

- users 1 to 0..1 merchant_profiles; users 1 to 0..1 carts; users 1 to many orders, notifications, disputes (as raiser).
- categories 1 to many categories (parent to children) and 1 to many products.
- merchant_profiles 1 to many products and 1 to many fulfillment_orders.
- products 1 to 1 inventory; products 1 to many cart_items, order_items and inventory_reservations.
- carts 1 to many cart_items.
- orders 1 to many fulfillment_orders, payments and inventory_reservations.
- fulfillment_orders 1 to many order_items and 1 to many disputes.

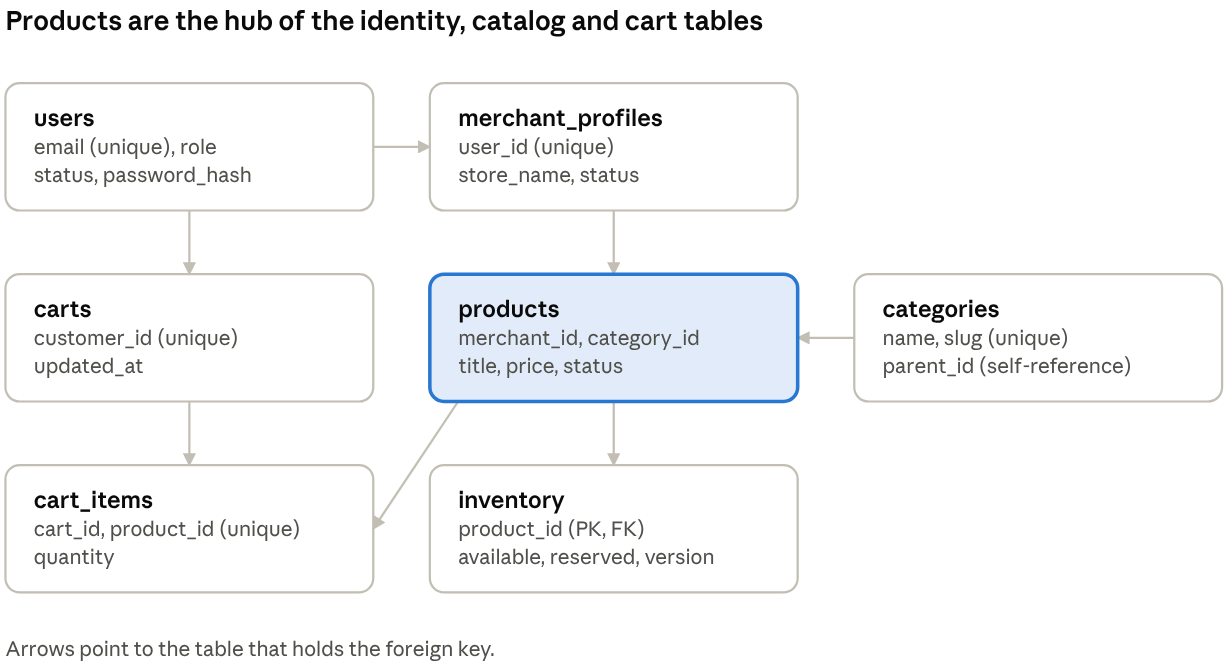
6.3 Indexes

Index	Purpose
products (merchant_id)	Storefront and merchant product lists
products (category_id, status)	Category browsing
products (lower(title)) or pg_trgm GIN index	Search
orders (customer_id, created_at DESC)	Customer order history

Index	Purpose
orders (status, expires_at)	Scheduler scan for abandoned orders
fulfillment_orders (merchant_id, status)	Merchant dashboard
inventory_reservations (order_id, status)	Release and commit
audit_logs (entity_type, entity_id)	Audit history lookup
notifications (user_id, created_at DESC)	Notification history

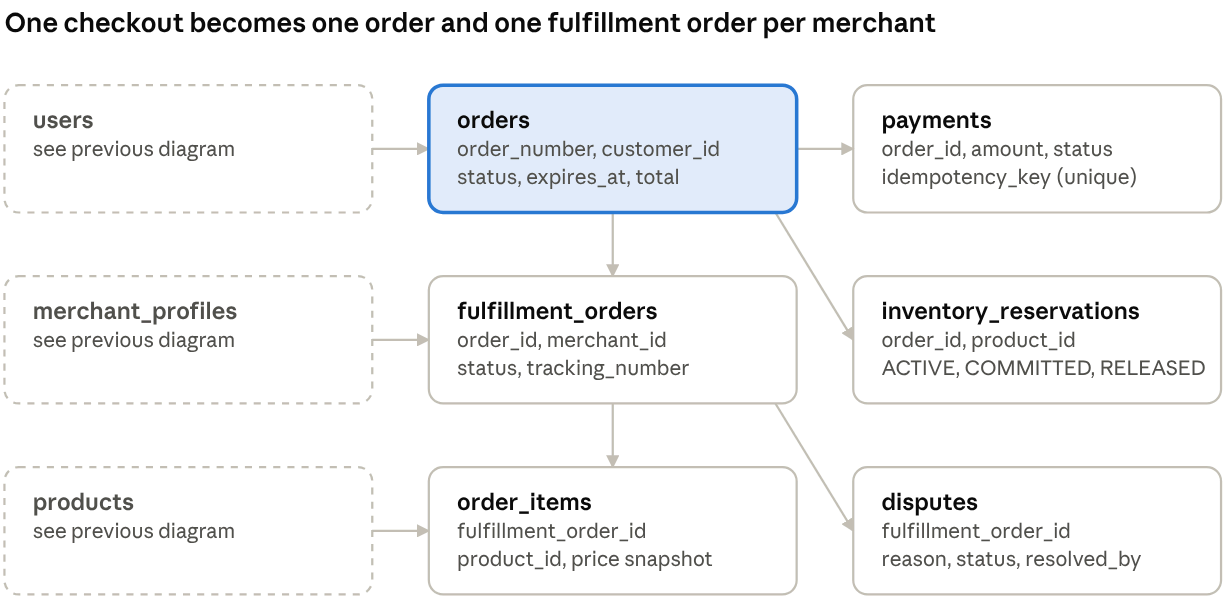
In addition, all UNIQUE constraints above create indexes automatically.

6.4 Entity-Relationship Diagram



ER diagram 1 · identity, catalog and cart · 7 tables

Diagram 1. Users own at most one merchant profile and one cart. Merchants and categories both feed products, and each product has exactly one inventory row.



ER diagram 2 · orders, fulfillment and payments · 6 tables plus 3 references

Diagram 2. A checkout creates one order, which owns its payments, its stock reservations and one fulfillment order per merchant. Fulfillment orders hold the order items and any disputes.

7. REST API Overview

All endpoints are prefixed with `/api/v1`. Unless marked Public, a valid JWT is required. Ownership rules (a merchant only touches own data, a customer only own orders) are enforced in the service layer in addition to role checks.

Method	Path	Role	Purpose
POST	/auth/register	Public	Register as CUSTOMER or request MERCHANT
POST	/auth/login	Public	Obtain JWT
GET	/admin/users	ADMIN	List and filter users

Method	Path	Role	Purpose
PATCH	/admin/users/{id}/status	ADMIN	Suspend or reactivate a user
POST	/merchants/apply	Authenticated	Create merchant profile (status PENDING)
GET	/admin/merchants	ADMIN	List merchant applications
PATCH	/admin/merchants/{id}/review	ADMIN	Approve or reject
GET	/categories	Public	Category tree
POST, PUT, DELETE	/admin/categories	ADMIN	Manage categories
GET	/products	Public	Search, filter, sort, paginate
GET	/products/{id}	Public	Product detail with availability
GET	/merchants/{id}/products	Public	Storefront
POST	/merchant/products	MERCHANT	Create product with initial stock
PUT	/merchant/products/{id}	MERCHANT	Update own product (price changes audited)
PATCH	/merchant/products/{id}/stock	MERCHANT	Adjust available stock
PATCH	/admin/products/{id}/status	ADMIN	Moderate (hide or remove)
GET	/cart	CUSTOMER	View cart with totals
POST	/cart/items	CUSTOMER	Add item

Method	Path	Role	Purpose
PUT, DELETE	/cart/items/{productId}	CUSTOMER	Change quantity or remove
POST	/orders/checkout	CUSTOMER	Reserve stock and create PENDING order
POST	/orders/{id}/payment	CUSTOMER	Pay (requires Idempotency-Key header)
GET	/orders, /orders/{id}	CUSTOMER	History and tracking
POST	/orders/{id}/cancel	CUSTOMER	Cancel while PENDING
GET	/merchant/fulfillments	MERCHANT	Dashboard filtered by status
PATCH	/merchant/fulfillments/{id}/process	MERCHANT	PAID to PROCESSING
PATCH	/merchant/fulfillments/{id}/ship	MERCHANT	PROCESSING to SHIPPED with tracking number
PATCH	/merchant/fulfillments/{id}/deliver	MERCHANT, ADMIN	SHIPPED to DELIVERED
POST	/disputes	CUSTOMER	Open a dispute
GET, PATCH	/admin/disputes, /admin/disputes/{id}/resolve	ADMIN	Review and resolve
POST	/admin/orders/{id}/cancel	ADMIN	Cancel with reason (audited)
GET	/admin/audit-logs	ADMIN	Filter by entity, actor, date
GET	/admin/metrics	ADMIN	Revenue, orders by status, top merchants

Method	Path	Role	Purpose
GET	/notifications	Authenticated	Own notification history
GET	/actuator/health, /actuator/metrics	ADMIN or internal	Operations

8. Git Workflow and Commit Conventions

8.1 Branching Strategy

The project uses a lightweight GitHub Flow, which suits a solo portfolio project and still looks professional:

- `main` is always buildable and protected: no direct pushes, merge through pull requests only, CI must pass.
- Every ticket gets one short-lived branch created from `main`, merged by squash or rebase, then deleted.
- Releases are marked with annotated semantic-version tags (`v0.1.0` after the foundation milestone, `v1.0.0` when all P0 and P1 tickets are done).

Branch naming: `<type>/ECOM-<number>-<short-kebab-description>`

Example	Meaning
`feature/ECOM-31-multi-vendor-checkout`	New functionality
`fix/ECOM-27-reservation-race-condition`	Bug fix
`chore/ECOM-3-docker-compose-postgres`	Tooling, config, dependencies
`docs/ECOM-57-readme-and-diagrams`	Documentation only
`test/ECOM-53-checkout-integration-test`	Tests only
`refactor/ECOM-32-order-mapper`	Behavior-preserving change

8.2 Commit Message Format (Conventional Commits)

```
<type>(<scope>): <subject>

<body: what and why, wrapped at 72 characters>

<footer: Refs ECOM-31 / Closes #31 / BREAKING CHANGE: ...>
```

Types

Type	Use for
feat	A new user-visible capability or endpoint
fix	A bug fix
refactor	Code change that neither fixes a bug nor adds a feature
test	Adding or correcting tests
docs	README, diagrams, API docs, comments only
build	Maven, Dockerfile, dependencies
ci	GitHub Actions and pipeline configuration
chore	Housekeeping that fits nowhere else (gitignore, formatting)
perf	Performance improvement (indexes, caching, query tuning)
style	Formatting only, no logic change

Scopes (match the package names): `auth`, `user`, `merchant`, `catalog`, `inventory`, `cart`, `order`, `payment`, `notification`, `audit`, `admin`, `scheduler`, `security`, `db`, `config`, `docker`, `deps`.

Rules

1. Subject in the imperative mood, lower case, no trailing period, at most 72 characters ("add", not "added" or "adds").
2. One logical change per commit; the build and tests pass at every commit.

3. Always reference the ticket in the footer (`Refs ECOM-31`); use `Closes #<issue>` on the final commit of a PR so GitHub closes the issue.
4. The body explains why, not what; the diff already shows what.
5. Breaking API or schema changes carry `!` after the scope and a `BREAKING CHANGE:` footer.
6. Never commit secrets, `.env` files or build output.
7. Database migrations are committed together with the entity change that needs them and are never edited after merge; add a new migration instead.

Examples

```
feat(inventory): reserve stock atomically with conditional update

Use a single UPDATE guarded by available_quantity >= :qty so two
concurrent checkouts can never oversell the last unit.

Refs ECOM-26
```

```
fix(scheduler): release reservations when cancelling expired orders

Refs ECOM-42
```

```
feat(order)!: rename order status SUCCESS to PAID

BREAKING CHANGE: clients must read PAID instead of SUCCESS.
Refs ECOM-30
```

```
test(cart): cover quantity above available stock
docs(readme): add checkout sequence diagram
build(docker): add multi-stage Dockerfile with layered jar
ci: run tests with Testcontainers on every pull request
chore(db): add flyway migration V4 for audit_logs
```

8.3 Pull Requests, Issues and Releases

- **PR title** follows the same Conventional Commit format as the squash commit (`feat(order): split checkout into fulfillment orders`).
- **PR template:** summary, linked ticket, what changed, how it was tested, screenshots or sample requests, checklist (tests added, migration added, docs updated, no secrets).

- **Issue labels:** type (`feature` , `bug` , `chore` , `docs`), priority (`P0` , `P1` , `P2`), epic (`epic:auth` , `epic:catalog` , ...), status via a project board (Backlog, Ready, In progress, In review, Done).
- **Milestones** on GitHub mirror section 9: M1 to M5.
- **Releases:** semantic versioning, a changelog generated from commit types (feat becomes minor, fix becomes patch, breaking becomes major).
- **Recommended hygiene:** signed commits are optional; a `CONTRIBUTING.md` that links this section signals maturity to recruiters.

9. Ticket Backlog

58 tickets grouped into 15 epics and 5 milestones. Each ticket maps to one branch (`type/ECOM-n-...`) and one pull request.

Legend. Priority: P0 = must have, P1 = should have, P2 = stretch. Points: 1 (about an hour) to 8 (about a day or more). Course = the course topic practiced (section 2).

Milestones

Milestone	Tickets	Outcome	Tag
M1 Foundation and identity	ECOM-1 to ECOM-12	Running app, Docker DB, CI, error handling, register and login with JWT	v0.1.0
M2 Catalog and inventory	ECOM-13 to ECOM-27	Merchants onboard, list products, customers search; stock is concurrency-safe	v0.2.0
M3 Cart and checkout	ECOM-28 to ECOM-35	Cart, multi-vendor checkout, simulated payment	v0.3.0
M4 Fulfillment and automation	ECOM-36 to ECOM-46	Merchant flow, async notifications, scheduler, audit logging	v0.4.0
M5 Admin and production	ECOM-47 to ECOM-58	Admin tools, full test suite, Actuator, Docker, docs	v1.0.0

Epic A: Project Setup and Foundations (M1)

ECOM-1 · Initialize project and package structure · Task · P0 · 2 pts · Course 1.1 Generate the Spring Boot project and create the package-by-feature skeleton from section 4.2.

- App starts and returns 404 on unknown paths; `mvn verify` passes.
- Packages exist with a `package-info` or placeholder class describing each feature.
- `.gitignore`, `README` stub, `LICENSE` and `CONTRIBUTING.md` committed.

ECOM-2 · Externalized configuration and profiles · Task · P0 · 2 pts · Course 1.3 Introduce dev, test and prod profiles and typed configuration.

- `@ConfigurationProperties` classes for JWT (secret, expiry), checkout (reservation minutes) and scheduler (interval).
- Secrets come from environment variables, never from committed files.
- Startup fails fast with a clear message if the JWT secret is missing in prod.

ECOM-3 · PostgreSQL via Docker Compose and Flyway baseline · Task · P0 · 3 pts · Course 7.1 Provide a reproducible local database and versioned migrations.

- `docker compose up db` starts PostgreSQL with a named volume.
- Flyway runs `V1__init.sql` on startup; Hibernate `ddl-auto` is `validate`.
- Migration naming rule documented in README.

ECOM-4 · Global exception handling and error format · Story · P0 · 3 pts · Course 2.4 One `@ControllerAdvice` produces a consistent JSON error body.

- Body contains timestamp, status, error code, message, path and a list of field errors.
- Mapped exceptions: validation (400), not found (404), forbidden (403), conflict and illegal state transition (409), unexpected (500 without stack trace).
- Domain exceptions (`ResourceNotFoundException`, `InsufficientStockException`, `IllegalStateTransitionException`) exist and are unit tested.

ECOM-5 · Logging foundation with correlation id · Task · P1 · 3 pts · Course 5.1 Structured, traceable logs across layers.

- A filter puts a correlation id into MDC and returns it in an `X-Request-Id` response header.
- Log pattern includes the correlation id; business events log key=value pairs (orderId, merchantId).
- No passwords, tokens or card-like data ever logged.

ECOM-6 · CI pipeline with GitHub Actions · Task · P1 · 2 pts · Course 7.1 Build and test every push and pull request.

- Workflow runs `mvn verify` on Java 21 with Maven cache.

- Branch protection requires the check to pass before merge.
- Build status badge in README.

Epic B: Authentication and Users (M1)

ECOM-7 · User entity and repository · Task · P0 · 2 pts · Course 3.1, 3.2

- `users` migration, `User` entity with role and status enums, `findByEmail` derived query.
- Email uniqueness enforced by DB constraint and case-insensitive lookup.
- `@DataJpaTest` covers save, lookup and duplicate email.

ECOM-8 · Registration endpoint · Story · P0 · 3 pts · Course 2.1, 2.2, 2.3, 4.2 `POST /auth/register` creates a CUSTOMER (admins are seeded, never self-registered).

- Request DTO validated (email format, password length and strength rule, name not blank); response DTO never exposes the hash.
- Duplicate email returns 409 with the standard error body.
- Custom validator annotation for the password policy.

ECOM-9 · Spring Security configuration and password hashing · Task · P0 · 3 pts · Course 5.3

- Stateless filter chain; CSRF disabled for the API with the reason documented.
- BCrypt `PasswordEncoder` bean; public routes whitelisted; everything else authenticated.
- 401 and 403 return the standard JSON error body through custom handlers.

ECOM-10 · JWT login · Story · P0 · 3 pts · Course 5.4 `POST /auth/login` returns a signed, expiring token.

- Token carries subject (user id) and role claim; expiry from configuration.
- Wrong credentials and suspended accounts get a generic 401 message (no user enumeration).
- Unit tests for token creation and parsing, including expired and tampered tokens.

ECOM-11 · JWT filter and role-based authorization · Story · P0 · 5 pts · Course 5.3, 5.4

- `OncePerRequestFilter` validates the bearer token and sets the security context.
- Method security (`@PreAuthorize`) protects role-specific endpoints; a `CurrentUser` helper exposes id and role to services.

- Tests prove CUSTOMER cannot call merchant or admin routes and anonymous users get 401.

ECOM-12 · Admin user management and seeded admin · Story · P1 · 3 pts · Course 2.1, 3.2

- `GET /admin/users` with pagination and role filter; `PATCH /admin/users/{id}/status` to suspend or reactivate.
- Initial admin created by a migration or a profile-guarded runner from environment variables.
- A suspended user cannot log in and existing tokens are rejected.

Epic C: Merchant Onboarding (M2)

ECOM-13 · Merchant application · Story · P0 · 3 pts · Course 3.1, 2.3 `POST /merchants/apply` creates a PENDING merchant profile for the logged-in user.

- Unique store name; one profile per user; validation on all fields.
- The user role becomes MERCHANT only after approval (ECOM-14).

ECOM-14 · Admin merchant review · Story · P0 · 3 pts · Course 2.1, 3.3

- List applications by status; approve or reject in one transaction that also updates the user role.
- A rejected applicant can re-apply; an approved merchant cannot be re-reviewed.
- Review recorded with reviewer id and timestamp, and an audit entry once ECOM-44 exists.

ECOM-15 · Merchant profile view and update · Story · P1 · 2 pts · Course 2.1, 2.3

- `GET` and `PUT /merchants/me` let an approved merchant read and edit own store name and description.
- Store name stays unique; a name change is audit-logged once ECOM-44 exists.

Epic D: Catalog (M2)

ECOM-16 · Category entity and admin CRUD · Story · P0 · 3 pts · Course 3.1

- Self-referencing `parent_id`; slug generated and unique; deleting a category with children or products is rejected with 409.
- Cycles prevented (a category cannot become its own descendant).

ECOM-17 · Category tree endpoint · Story · P1 · 2 pts · Course 2.2, 4.2

- `GET /categories` returns a nested tree DTO built with a single query, not one query per node.
- Cacheable (see ECOM-22 for the cache strategy).

ECOM-18 · Create product with initial stock · Story · P0 · 5 pts · Course 2.1, 2.3, 3.1, 4.2
`POST /merchant/products` by an approved merchant.

- Product and inventory rows are created in one transaction.
- Validation: title length, price greater than zero with at most 2 decimals, quantity zero or more, category exists.
- Product is immediately visible in public search; the merchant id comes from the token, never from the request body.

ECOM-19 · Update and hide own product · Story · P0 · 3 pts · Course 5.3, 2.4

- `PUT /merchant/products/{id}` updates fields; attempting to touch another merchant product returns 404 (not 403, to avoid leaking existence).
- A merchant can set status HIDDEN or ACTIVE; REMOVED is admin-only.
- Price change publishes a domain event or calls the audit hook (completed in ECOM-45).

ECOM-20 · Public product browse and detail · Story · P0 · 3 pts · Course 2.1, 4.2

- `GET /products/{id}` returns merchant summary, category path and availability flag (in stock or not), never raw reserved counts.
- Only ACTIVE products from APPROVED, non-suspended merchants are visible.

ECOM-21 · Search, filter and pagination · Story · P0 · 5 pts · Course 3.2, 2.1 `GET /products?q=&category=&minPrice=&maxPrice=&merchantId=&sort=&page=&size=`.

- Combine filters with Spring Data Specifications or a `@Query`; size capped at a maximum (for example 50).
- Sorting whitelist (price, createdAt, title) to prevent arbitrary column sorting.
- Repository test with realistic data set; query uses the indexes from section 6.3.

ECOM-22 · Cache popular searches and category tree · Story · P1 · 3 pts · Course 6.3

- `@Cacheable` on search with a key built from all parameters; short TTL configured in properties.
- Eviction on product create, update, status change and stock reaching zero.
- Test proves second identical call does not hit the repository, and that an update invalidates the entry.

ECOM-23 · Merchant storefront endpoint · Story · P1 · 2 pts · Course 2.1

- `GET /merchants/{id}/products` lists a seller entire active catalog, paginated.
- Unknown or unapproved merchant returns 404.

ECOM-24 · Admin product moderation · Story · P1 · 2 pts · Course 5.3, 6.2

- `PATCH /admin/products/{id}/status` with a reason; removed products disappear from search and carts show them as unavailable.
- Action is audit-logged.

Epic E: Inventory (M2)

ECOM-25 · Stock adjustment by merchant · Story · P0 · 2 pts · Course 2.1, 3.3

- `PATCH /merchant/products/{id}/stock` sets or increments available stock; result can never be negative.
- Only available_quantity changes; units reserved by pending orders are never touched, and the change is audit-logged.

ECOM-26 · Atomic reserve, release and commit service · Story · P0 · 8 pts · Course 1.2, 3.3
The core of overselling prevention (section 5.2).

- `InventoryService` exposes `reserve`, `release` and `commit`, each transactional and idempotent per reservation id.
- Reserve uses the conditional UPDATE (or pessimistic lock) and processes products in ascending id order.
- Zero updated rows throws `InsufficientStockException` carrying the product id.
- Unit tests for the happy path and each failure branch.

ECOM-27 · Concurrency test for overselling · Task · P0 · 3 pts · Course 5.2, 3.3

- Integration test starts N threads that each try to reserve the last units; exactly the available quantity succeeds.
- Run against real PostgreSQL via Testcontainers.
- Documented in the README as evidence of correctness.

Epic F: Shopping Cart (M3)

ECOM-28 · Cart and item management · Story · P0 · 5 pts · Course 2.1, 2.3, 3.1

- Cart created lazily per customer; `POST /cart/items`, `PUT` and `DELETE /cart/items/{productId}`.

- Adding the same product increases quantity; quantity must be 1 or more and within a sane maximum.
- Inactive, removed or out-of-stock products cannot be added (409 with clear code).

ECOM-29 · View cart with totals and availability · Story · P0 · 3 pts · Course 2.2, 4.2

- `GET /cart` returns lines grouped by merchant, current prices, line totals, cart total and a per-line availability warning.
- Uses a fetch join or entity graph so loading the cart is not N+1.

Epic G: Orders and Checkout (M3)

ECOM-30 · Order entities and state machine · Story · P0 · 5 pts · Course 3.1, 2.4

- Entities `Order`, `FulfillmentOrder`, `OrderItem`, migrations and enums.
- A single `OrderStateMachine` (or enum with allowed-transition map) is the only place transitions are validated; the table in section 5.1 is its test specification.
- Illegal transitions throw `IllegalStateTransitionException` (409). Parameterized unit tests cover every allowed and every forbidden pair.

ECOM-31 · Multi-vendor checkout · Story · P0 · 8 pts · Course 3.3, 1.2, 7.2 `POST /orders/checkout` implements the algorithm in section 5.3 in one transaction.

- Reserves stock, creates one Order and one FulfillmentOrder per merchant, snapshots title and price, records reservations, clears the cart, sets `expires_at` to now plus 15 minutes.
- If any line cannot be reserved, nothing is persisted and the 409 response lists the failing products.
- Response contains the order number, per-merchant breakdown and the payment deadline.
- Integration test: two merchants, three products, verifies all rows and stock counters.

ECOM-32 · Customer order history and tracking · Story · P0 · 3 pts · Course 2.2, 4.2, 3.2

- `GET /orders` (paginated, newest first) and `GET /orders/{id}` showing each fulfillment order status, tracking number and carrier.
- A customer asking for someone else order receives 404.
- Mapper converts entities to DTOs; entities never reach the controller.

ECOM-33 · Customer cancels a pending order · Story · P1 · 3 pts · Course 3.3

- `POST /orders/{id}/cancel` allowed only while PENDING.

- Reservations are released in the same transaction; reason CUSTOMER stored; audit entry written.
- Cancelling a PAID order returns 409 (handled by admin or dispute flow).

Epic H: Payment (M3)

ECOM-34 · Simulated payment gateway · Story · P0 · 3 pts · Course 1.2, 1.3

- `PaymentGateway` interface with a `SimulatedPaymentGateway` implementation selected by configuration (always succeed, always fail, or fail when amount ends in a configured value).
- Returns a provider reference; the gateway is injected, so tests substitute a mock.
- Adds artificial latency only in the dev profile.

ECOM-35 · Payment flow with idempotency · Story · P0 · 8 pts · Course 3.3, 7.2 `POST /orders/{id}/payment` with an `Idempotency-Key` header.

- Success: Payment SUCCEEDED, Order and all FulfillmentOrders PAID, reservations COMMITTED (stock permanently deducted), confirmation notification queued, all in one transaction.
- Failure: Payment FAILED recorded, order stays PENDING and can be retried until it expires.
- Repeating the same key returns the original result without a second charge; paying an expired or cancelled order returns 409.
- Concurrency case: payment and scheduler cancellation racing on the same order cannot both win (use a status-conditional update or optimistic version).

Epic I: Merchant Fulfillment (M4)

ECOM-36 · Merchant order dashboard · Story · P0 · 3 pts · Course 3.2, 2.1

- `GET /merchant/fulfillments?status=` lists only own fulfillment orders with items and customer shipping reference, paginated.
- Default filter shows PAID orders waiting to be processed; counts per status available for the dashboard header.

ECOM-37 · Mark fulfillment as PROCESSING · Story · P0 · 2 pts · Course 2.1, 5.3

- Allowed only from PAID and only by the owning merchant; goes through the state machine.
- Another merchant receives 404, wrong state receives 409.

ECOM-38 · Mark fulfillment as SHIPPED with tracking number · Story · P0 · 3 pts · Course 2.3, 6.4

- Request requires tracking number (custom validator for allowed characters and length) and optional carrier.
- Sets `shipped_at`; queues a shipping notification containing the tracking details.

ECOM-39 · Mark fulfillment as DELIVERED · Story · P1 · 2 pts · Course 2.1

- Allowed from SHIPPED by the owning merchant or an admin; sets `delivered_at`.
- When every fulfillment order of an order is DELIVERED, the order summary shows as completed.

Epic J: Asynchronous Notifications (M4)

ECOM-40 · Async executor and notification service · Story · P0 · 3 pts · Course 6.4, 1.2

- `@EnableAsync` with a named, bounded `ThreadPoolTaskExecutor` bean and a rejection policy.
- `NotificationService` persists a QUEUED notification, then sends it asynchronously through an `EmailSender` interface whose simulation implementation logs the email and marks it SENT or FAILED.
- Failures in sending never roll back or slow the business transaction.

ECOM-41 · Notification triggers and history endpoint · Story · P1 · 3 pts · Course 6.4, 2.1

- Triggers: order confirmation (on PAID), shipping update (on SHIPPED), cancellation (customer, admin or timeout).
- Notifications are created only after the surrounding transaction commits (`@TransactionalEventListener(AFTER_COMMIT)`).
- `GET /notifications` returns the current user own messages, newest first.

Epic K: Scheduled Maintenance (M4)

ECOM-42 · Abandoned order cleanup job · Story · P0 · 5 pts · Course 6.4, 3.2, 3.3
Implements the flow in section 5.4.

- `@Scheduled` job every 15 minutes (interval from configuration) finds PENDING orders whose `expires_at` has passed using the `(status, expires_at)` index.
- Each order is cancelled in its own transaction (reason PAYMENT_TIMEOUT), reservations released, audit actor SYSTEM, notification queued.

- A failure on one order is logged and does not stop the batch; the job logs how many orders it cancelled.
- Test uses the injectable Clock to prove orders younger than 15 minutes are untouched.

ECOM-43 · Scheduler hardening · Task · P2 · 3 pts · Course 6.4

- Batch size limit per run; metrics counter for cancelled orders exposed through Actuator.
- Optional: ShedLock so only one application instance runs the job in a multi-instance deployment.

Epic L: Audit Logging (M4)

ECOM-44 · Audit log entity and `@Audited` aspect · Story · P0 · 5 pts · Course 6.2

- Custom `@Audited(action, entityType)` annotation and an `@Aspect` that records actor, role, action, entity id and before and after values as JSON.
- Audit rows are written in the same transaction as the change so they cannot diverge; the table is append-only (no update or delete methods in the repository).
- Unit test proves a failed business transaction leaves no audit row.

ECOM-45 · Audit price changes and order cancellations · Story · P0 · 3 pts · Course 6.2

- Product price change logs old and new price with the merchant as actor; only real changes are logged.
- Order cancellation logs who cancelled (customer, admin or SYSTEM) and why.
- Also applied to merchant review, product moderation and dispute resolution.

ECOM-46 · Audit log query endpoint · Story · P1 · 2 pts · Course 3.2, 5.3

- `GET /admin/audit-logs` filtered by entity type, entity id, actor and date range, paginated.
- ADMIN only; the endpoint itself is read-only.

Epic M: Administration (M5)

ECOM-47 · Admin cancels an order · Story · P1 · 3 pts · Course 3.3, 6.2

- `POST /admin/orders/{id}/cancel` with a mandatory reason; allowed for PENDING, PAID and PROCESSING fulfillment orders, not SHIPPED or later.

- Policy documented: reservations are released if PENDING; for paid orders not yet shipped, committed units return to available stock and the payment is marked for refund (simulated).
- Audit entry and customer notification produced.

ECOM-48 · Disputes · Story · P2 · 5 pts · Course 2.1, 3.1

- Customer opens a dispute on own fulfillment order (`POST /disputes`); admin lists and resolves with a note (`PATCH /admin/disputes/{id}/resolve`).
- Only one OPEN dispute per fulfillment order; resolution is audit-logged.

ECOM-49 · Platform metrics endpoint · Story · P1 · 5 pts · Course 3.2

- `GET /admin/metrics` returns total revenue (PAID and later), order counts by status, top 5 merchants by revenue and orders per day for a date range.
- Implemented with `@Query` aggregates returning projections, not by loading entities into memory.

Epic N: Testing (M5, written alongside features, closed here)

ECOM-50 · Service-layer unit tests · Task · P0 · 5 pts · Course 5.2

- Mockito tests for checkout, payment, cancellation, state machine, cleanup job and audit aspect.
- Targets: all business branches covered; line coverage of the service packages at 80 percent or more (JaCoCo report in CI).

ECOM-51 · Web-layer tests · Task · P1 · 3 pts · Course 5.2, 2.3, 2.4

- `@WebMvcTest` slices for every controller: status codes, validation errors, error body shape, role-based access.

ECOM-52 · Repository tests · Task · P1 · 3 pts · Course 5.2, 3.2

- `@DataJpaTest` for search specifications, the abandoned-order query, aggregate queries and unique constraints, against PostgreSQL via Testcontainers.

ECOM-53 · End-to-end checkout integration test · Task · P0 · 5 pts · Course 5.2, 7.2

- `@SpringBootTest` with a random port walks the whole story: register merchant and customer, approve merchant, create products, add to cart, checkout across two merchants, pay, process, ship, deliver.
- A second scenario lets the order expire and verifies the scheduler restores stock.
- The test doubles as the executable version of the walkthrough in section 12.

Epic O: Production Readiness (M5)

ECOM-54 · Actuator, health and metrics · Task · P1 · 3 pts · Course 6.1

- Expose health, info and metrics; secure everything except health; liveness and readiness probes enabled.
- Custom health indicator (for example, failed notifications in the last hour) and custom counters (orders created, payments failed).

ECOM-55 · OpenAPI documentation · Task · P1 · 2 pts · Course 2.1

- Swagger UI with JWT bearer authorization, grouped by role, request and response examples for checkout and payment.

ECOM-56 · Dockerize the application · Task · P0 · 3 pts · Course 7.1

- Multi-stage Dockerfile (build then slim runtime, non-root user, layered jar).
- `docker compose up` starts app and database with a health-gated start order; configuration via environment variables.

ECOM-57 · Portfolio documentation and seed data · Task · P0 · 3 pts · Course 7.2

- README per section 15, architecture and ER diagrams, checkout sequence diagram, Postman or HTTP-file collection, and a `dev` profile seeder creating sample merchants, products and customers.
- Short demo script (curl commands) that reproduces the full order lifecycle in under two minutes.

ECOM-58 · Performance and query review · Task · P2 · 3 pts · Course 3.2, 6.3

- Enable SQL logging in test, find and remove N+1 queries in cart, order history and storefront.
- Verify indexes with `EXPLAIN` for search, scheduler scan and merchant dashboard; record before and after in the README.

10. Testing Strategy

Level	Tooling	What it proves	Tickets
Unit	JUnit 5, Mockito, injectable Clock	Business rules, state machine, reservation logic in isolation	ECOM-50

Level	Tooling	What it proves	Tickets
Web slice	`@WebMvcTest`, MockMvc, Spring Security test	Routing, validation, status codes, error body, role access	ECOM-51
Persistence slice	`@DataJpaTest`, Testcontainers PostgreSQL	Queries, constraints, migrations match entities	ECOM-52
Concurrency	Testcontainers, `ExecutorService`	No overselling under parallel checkout	ECOM-27
End to end	`@SpringBootTest`, random port	Real flows across all layers including the scheduler	ECOM-53

Rules of thumb: test behavior rather than implementation, one assertion theme per test, descriptive names (`checkout_rollsBackAllReservations_whenOneItemOutOfStock`), no sleeping (control time through the Clock bean), and every bug fix starts with a failing test.

11. Definition of Done and Quality Gates

A ticket is done only when all of the following are true:

- Acceptance criteria met and demonstrated (sample request or test).
- Tests added at the right level; CI is green.
- Database changes delivered as a new Flyway migration; schema doc in section 6 updated if the model changed.
- Input validated, authorization checked, ownership enforced where relevant.
- No entity exposed from a controller; no business logic in a controller; no repository call from a controller.
- Meaningful log lines for business events, no sensitive data logged.
- Commit messages follow section 8; PR description filled in; branch deleted after merge.
- README or OpenAPI updated when the public behavior changed.

12. End-to-End Walkthrough: One Checkout

This is the course capstone flow (course topic 7.2), from HTTP request to database rows and back.

```
POST /api/v1/orders/checkout   (Bearer JWT, role CUSTOMER)
 1. RequestIdFilter        -> correlation id into MDC
 2. JwtAuthFilter          -> authenticates, sets SecurityContext
 3. OrderController        -> no body validation needed, calls
checkoutService.checkout(currentUser)
 4. CheckoutService @Transactional
      a. cartRepository    -> load cart with items (fetch join)
      b. inventoryService.reserve(...)  -> conditional UPDATE per product,
ids ascending
      c. build Order + FulfillmentOrder per merchant + OrderItem snapshots
      d. orderRepository.save, reservationRepository.saveAll (ACTIVE)
      e. cart cleared; expires_at = now + 15 min
 5. COMMIT                 -> any failure rolls back stock, order and
reservations together
 6. OrderMapper            -> entity to OrderResponse DTO
 7. Jackson                -> JSON 201 Created

POST /api/v1/orders/{id}/payment   (Idempotency-Key: abc-123)
 1. PaymentService @Transactional -> idempotency lookup, gateway call
 2. success: Payment SUCCEEDED, statuses PAID, reservations COMMITTED
 3. publish OrderPaidEvent
 4. AFTER_COMMIT listener -> NotificationService (@Async) -> email simulation

Every 15 minutes: AbandonedOrderJob -> cancel expired PENDING orders ->
release stock
```

13. Suggested Build Order

Week	Focus	Tickets
1	Setup, config, Docker, CI, error handling, logging	ECOM-1 to ECOM-6
2	Users, security, JWT, merchants	ECOM-7 to ECOM-14
3	Catalog, search, caching	ECOM-16 to ECOM-24
4	Inventory and cart	ECOM-25 to ECOM-29
5	Orders, checkout, payment	ECOM-30 to ECOM-35
6	Fulfillment, async, scheduler	ECOM-36 to ECOM-43

Week	Focus	Tickets
7	Audit, admin, metrics	ECOM-44 to ECOM-49
8	Tests, Actuator, Docker, docs, polish	ECOM-50 to ECOM-58

Write the tests of each ticket in the same PR as the feature; ECOM-50 to ECOM-53 close gaps rather than start from zero. Cut P2 tickets first if time is short.

14. Key Design Decisions and Risks

Topic	Decision	Reason or risk
Overselling	Conditional UPDATE on inventory plus CHECK constraints	Correct under concurrency without long locks; verified by ECOM-27
Order model	Parent order plus fulfillment orders per merchant	Matches the spec requirement to split one checkout; lets each merchant progress independently
Money	BigDecimal and NUMERIC, price snapshots	Avoids rounding errors and history rewrites
Payment	Idempotency key plus unique constraint	Safe retries; no double charges
Payment versus timeout race	Status-conditional update or `@Version`	Prevents an order being both paid and cancelled
Notifications	AFTER_COMMIT event, `@Async`, persisted status	Never notify about a rolled-back change; user never waits
Audit	Same transaction, append-only	Cannot diverge from the change it describes
Cache	Short TTL plus explicit eviction	Stale search results are acceptable for seconds, not minutes
Scheduler on many instances	ShedLock (stretch)	Without it each instance would run the job

Topic	Decision	Reason or risk
Scope creep	P2 tickets are optional	Protects the finish date

15. Portfolio Presentation Checklist

A recruiter reads the README for two minutes. Make those minutes count:

1. One-paragraph pitch and a feature list with the hard problems named (overselling, multi-vendor split, idempotent payments, scheduled recovery).
2. Architecture diagram, ER diagrams and the order state machine picture.
3. Quick start: `git clone`, `docker compose up`, open Swagger UI, sample login credentials for each role.
4. Demo script reproducing the full order lifecycle.
5. Tech stack table and a short section on design decisions (copy from section 14).
6. Test report: coverage badge, Testcontainers mention, the concurrency test result.
7. Link to the project board and the closed tickets, which show a real workflow.
8. Clean Git history following the conventions of section 8, release tags and a changelog.

16. Possible Future Improvements

- Real payment provider integration with webhooks and refunds.
- Outbox pattern and a message broker for notifications and events.
- Refresh tokens and token revocation; rate limiting on login.
- Product variants, images and reviews; full-text search with PostgreSQL trigram or an external search engine.
- Redis cache for multi-instance deployments; ShedLock or a job scheduler for clustering.
- Observability stack: Prometheus, Grafana and distributed tracing.